ОТЧЁТ

о разработке RAG-системы для документации ViPNet Coordinator HW 5

Параметр	Значение
Дата	25 февраля 2026
Репозиторий	https://github.com/maratsaratov/tech-task_infotecs
Исходные данные	6 документов, ~1 200 страниц
Retrieval-метод	Hybrid BM25 + bge-m3
LLM-генерация	Qwen2.5-7B-Instruct (GGUF / CPU-offload)

Оглавление

1. Введение ... 3
1.1 Постановка задачи ... 3
1.2 Используемые технологии ... 3
2. Структура репозитория ... 4
2.1 Корневые файлы ... 4
2.2 Директория chunks_and_bechmark/ ... 4
2.3 Инструкция для запуска ноутбуков ... 5
3. Описание разработанной системы ... 6
3.1 Архитектура ... 6
3.2 Иерархический чанкинг (Parent-Child) ... 6
3.2.1 Функция process_doc() — обработка документа ... 6
3.2.2 Типы блоков и стратегия разбиения ... 7
3.3 Гибридный поиск (Hybrid Retrieval) ... 7
3.3.1 Компоненты поиска ... 7
3.3.2 Функция retrieve_hybrid() и RRF-формула ... 8
3.3.3 Функция get_context() — получение родительских чанков ... 8
3.4 Модель эмбеддингов BAAI/bge-m3 ... 8
3.5 Языковая модель Qwen2.5-7B-Instruct ... 8
3.5.1 Три режима загрузки LLM ... 9
3.5.2 Системный промпт ... 9
4. Исследования и метрики ... 10
4.1 Метрики оценки ... 10
4.2 Результаты бенчмарка retrieval ... 10
4.3 Анализ по типам вопросов ... 10
4.4 Анализ по сложности вопросов ... 10
4.5 End-to-End оценка (10 вопросов) ... 11
5. Примеры реальных диалогов с системой ... 12
5.1 Определение понятия ... 12
5.2 Процедурный вопрос (пошаговые инструкции) ... 12
5.3 Вопрос вне области знаний системы ... 12
6. Описание входных данных ... 14
6.1 Корпус документов ... 14
6.2 QA-бенчмарк ... 14
7. Возможные улучшения ... 15
7.1 Retrieval ... 15
7.2 Генерация ... 15
7.3 Чанкинг ... 15
7.4 Оценка ... 15
8. Заключение ... 16

1. Введение

1.1 Постановка задачи

Документация программно-аппаратного комплекса ViPNet Coordinator HW 5 насчитывает шесть технических руководств общим объёмом более 1 200 страниц. Традиционный поиск требует последовательного просмотра нескольких документов, прежде чем оператор найдёт ответ на конкретный вопрос.

Поставленная задача посвящена разработке системы автоматических ответов на вопросы на основе архитектуры RAG (Retrieval-Augmented Generation). Система принимает вопрос на естественном языке и возвращает точный ответ с указанием источников.

> Цель проекта
> Создать RAG-систему, способную отвечать на вопросы по документации ViPNet Coordinator HW 5 с использованием открытых языковых моделей размером не более 8 миллиардов параметров.

1.2 Используемые технологии

Компонент	Технология	Назначение
Платформа	Google Colab (Tesla T4, 16 GB VRAM)	Среда обучения и тестирования
Язык	Python 3.11	Основной язык разработки
Эмбеддинги	BAAI/bge-m3	Многоязычные векторные представления
Разреженный поиск	BM25 (rank-bm25), TF-IDF (scikit-learn)	Лексический retrieval
LLM	Qwen2.5-7B-Instruct	Генерация ответов
GGUF-инференс	llama-cpp-python	Квантизованный запуск на T4
PDF → TXT	LlamaParse	Конвертация исходников

2. Структура репозитория

Все материалы проекта опубликованы в репозитории https://github.com/maratsaratov/tech-task_infotecs. Ниже приведено описание каждого файла и директории.

2.1 Корневые файлы

Файл / Директория	Описание
rag_vipnet_master.ipynb	Главный ноутбук: полный RAG-пайплайн — чанкинг, индексирование, retrieval, LLM-генерация, интерактивный чат. Содержит три режима загрузки LLM: GGUF (рекомендован для T4), CPU offload, малая модель 3B.
qa_dataset.json	QA-бенчмарк: 50 вручную составленных вопросов с эталонными ответами и gold_chunk_id. Используется для автоматической оценки качества retrieval и генерации.
01_txt.txt	ViPNet Coordinator HW 5. Подготовка к работе (~700 child-чанков). Инициализация устройства, роли администратора, мастер первоначальной настройки.
02_txt.txt	ViPNet Coordinator HW 5. Настройка через CLI (~800 child-чанков). Командный интерпретатор, конфигурация сети, система защиты от сбоев.
03_txt.txt	ViPNet Coordinator HW 5. Веб-интерфейс (~600 child-чанков). Управление через браузер, настройка параметров безопасности.
04_txt.txt	ViPNet Coordinator HW 5. Справочник команд CLI (~1 600 child-чанков, наибольший объём). Синтаксис всех команд, параметры, примеры использования.
06_txt.txt	ViPNet Coordinator HW 5. История версий (~400 child-чанков). Изменения по версиям продукта, changelog.
07_txt.txt	ViPNet Coordinator HW 5. Перечень совместимых трансиверов (~400 child-чанков). Аппаратные характеристики, таблицы совместимости.
README.md	Краткое описание репозитория.

2.2 Директория chunks_and_bechmark/

Директория содержит вспомогательные ноутбуки, полученные выборки в результате чанкинга и результаты бенчмарка, полученные в ходе разработки.

Файл	Описание
rag_benchmark_vipnet.ipynb	Изолированный ноутбук для бенчмарка retrieval-методов. Загружает готовые чанки, строит индексы четырёх методов (TF-IDF, BM25, Dense, Hybrid), вычисляет MRR / Recall@k / Context-F1, строит визуализации.
child_chunks.json	Сериализованные child-чанки корпуса (~4 500 объектов). Каждый чанк: chunk_id, parent_id, doc_id, section_path, block_type, text, text_with_context, char_count.
parent_chunks.json	Сериализованные parent-чанки (~3 900 объектов). Передаются в LLM при генерации ответа — содержат расширенный контекст до 4 096 символов.

all_chunks.json	Объединенная выборка из child- и parent-чанков (~8 400 объектов).

2.3 Инструкция для запуска ноутбуков

Для запуска ноутбука rag_benchmark_vipnet.ipynb необходимо, чтобы в одной директории с ноутбуком находились все файлы из директории chunks_and_bechmark/. Данный ноутбук используется для создания бенчмарка и исследования retrieval-методов.

Для запуска ноутбука rag_vipnet_master.ipynb необходимо, чтобы в одной директории с ноутбуком находились все корневые файлы. Данный ноутбук – главный, в нем содержится полный пайплайн - чанкинг, индексирование, retrieval, LLM-генерация, интерактивный чат.

3. Описание разработанной системы

3.1 Архитектура

Система реализована по классической архитектуре RAG с иерархической структурой чанков (Parent-Child) и двухэтапным поиском. Процесс обработки запроса включает четыре последовательных этапа.

Индексирование	Инференс
Чанкинг: Parent + Child уровни	Hybrid Retrieval (BM25 + Dense + RRF)
BM25-индекс (rank-bm25)	Загрузка parent-чанков из PARENT_MAP
Dense-индекс (BAAI/bge-m3)	Формирование промпта (top-5 parent-чанков)
Сохранение в JSON	LLM → Ответ + список источников

3.2 Иерархический чанкинг (Parent-Child)

Ключевой особенностью системы является двухуровневая структура хранения текста, которая разделяет задачи поиска и генерации. Этот подход решает фундаментальное противоречие RAG-систем: для точного поиска нужны короткие фрагменты с высокой плотностью ключевых слов, а для качественной генерации — длинный контекст, позволяющий LLM сформировать связный ответ.

> Принцип работы иерархического чанкинга:
> Child-чанки (~800 символов) — индексируются в BM25 и Dense, используются для поиска.
> Parent-чанки (до 4 096 символов) — хранятся в PARENT_MAP, передаются в LLM при генерации.
> Поиск ведётся по child-чанкам (точность), ответ генерируется по parent-чанкам (полнота контекста).

Параметр	Child-чанки (поиск)	Parent-чанки (генерация)
Назначение	Индексируются в BM25 и Dense	Передаются в промпт LLM
Целевой размер	~800 символов (~200 токенов)	До 4 096 символов (~1 024 токена)
Медиана	363 символа	~1 200 символов
Количество	~4 500	~3 900
Связь	parent_id указывает на родителя	child_ids — список дочерних

3.2.1 Функция process_doc() — обработка документа

Основная функция чанкинга принимает путь к TXT-файлу и конфигурацию документа, возвращает список объектов Chunk двух уровней. Алгоритм работы:

1. Очистка текста функцией clean_text(): удаление колонтитулов, номеров страниц, точечных строк оглавления с помощью регулярных выражений.
2. Удаление оглавления функцией remove_toc(): детектирование блока «строка + правовыровненный номер страницы», пропуск до первого содержательного заголовка.

3. Парсинг разделов функцией parse_sections(): определение уровней заголовков по шаблонам (Глава N., команды CLI, структурированные заголовки).
4. Построение иерархии путей функцией build_paths(): формирование section_path вида «Глава 2 > Настройка сети > Параметры DHCP».
5. Типизация блоков функцией det_bt(): классификация каждого раздела по типу содержимого.
6. Разбиение на child-чанки функцией split_children(): адаптивное разбиение по абзацам с целевым размером CHILD_TARGET_CHARS = 800 символов.
7. Создание объектов Chunk: parent и его дочерние child с установкой перекрёстных ссылок parent_id / child_ids.

3.2.2 Типы блоков и стратегия разбиения

Каждый чанк получает тип (block_type), определяющий стратегию разбиения и представление в промпте LLM:

Тип блока	Критерий определения	Стратегия разбиения
command	Блоки «Синтаксис», «Режимы командного интерпретатора» в справочнике команд	Разбивается по подразделам (синтаксис, параметры, примеры)
code_example	Приглашение hostname# или hostname> в тексте	Сохраняется целиком (примеры нельзя обрезать)
table	Три и более строк с выравниванием по пробелам (≥3 колонки)	Сохраняется целиком — таблицы неделимы
list	Три и более маркированных/нумерованных элементов (доля ≥30%)	Разбивается по группам элементов
config	Секции в квадратных скобках [section_name]	Разбивается по секциям
text	Всё остальное	Разбивается по абзацам

3.3 Гибридный поиск (Hybrid Retrieval)

Retrieval-компонент объединяет лексический и семантический поиск через алгоритм Reciprocal Rank Fusion (RRF), получая преимущества обоих подходов.

3.3.1 Компоненты поиска

Метод	Модель / Библиотека	Сильные стороны	Слабые стороны
TF-IDF baseline	scikit-learn, символьные n-граммы (3–5)	Нечувствителен к морфологии благодаря символьным n-граммам	Хуже BM25 на технических терминах
BM25	rank-bm25, k1=1.5, b=0.75	Точные CLI-команды, числа, аббревиатуры, «reasoning»-вопросы	Семантически близкие, но лексически разные запросы
Dense (bge-m3)	BAAI/bge-m3, 1 024-мерные эмбеддинги	Определения понятий, перефразированные запросы, многоязычность	Специфические технические термины без контекста

Hybrid RRF	BM25 + Dense + RRF (k=60)	Лучший MRR и Recall — объединяет преимущества обоих методов	Не устраняет семантические разрывы при отсутствии чанка

3.3.2 Функция retrieve_hybrid() и RRF-формула

Гибридный retrieval реализован в функции retrieve_hybrid(query, top_k=5). Алгоритм:

8. BM25 возвращает top-15 результатов с лексическими оценками.
9. Dense (bge-m3) возвращает top-15 результатов по косинусному сходству эмбеддингов.
10. Функция rrf_fusion() объединяет оба ранжирования по формуле RRF.
11. Возвращаются top-k итоговых результатов.

> Формула Reciprocal Rank Fusion:
> score(d) = Σ_i 1 / (k + rank_i(d)),
> где k = 60 (константа сглаживания), rank_i — позиция документа d в i-м ранжировании.
> RRF устойчив к различным масштабам оценок BM25 и косинусного сходства.

3.3.3 Функция get_context() — получение родительских чанков

После hybrid retrieval функция get_context() выполняет переход от child- к parent-чанкам: для каждого найденного child-чанка извлекается его родительский parent-чанк из PARENT_MAP. Дубликаты (несколько child от одного parent) фильтруются. В LLM передаются только уникальные parent-чанки.

3.4 Модель эмбеддингов BAAI/bge-m3

Для семантического поиска выбрана модель BAAI/bge-m3 как наилучший вариант для русскоязычной технической документации в категории открытых моделей.

Характеристика	Значение
Архитектура	XLM-RoBERTa Large (multi-task)
Размер модели	~2.3 GB (570M параметров)
Размер эмбеддинга	1 024 измерения
Поддерживаемые языки	100+ языков включая русский
Ключевые возможности	Dense retrieval, sparse retrieval, multi-vector (ColBERT)
Нормализация	normalize_embeddings=True (косинусное сходство через @-умножение)
Преимущество перед E5-small	MRR 0.549 vs 0.590* при идентичных условиях; лучше на definition-вопросах

** Разница объясняется разными параметрами чанкинга в разных экспериментах, а не качеством модели.*

3.5 Языковая модель Qwen2.5-7B-Instruct

Выбор LLM определялся требованием локального развёртывания на GPU Tesla T4 (16 GB VRAM) при параметрах не более 8B. Qwen2.5-7B-Instruct является наилучшим вариантом в этой категории по бенчмаркам с поддержкой русского языка.

Характеристика	Значение
Архитектура	Transformer decoder-only (Qwen2.5)
Параметры	7.6 млрд (7B)
Контекстное окно	128 000 токенов — без ограничений для нашего корпуса
VRAM (GGUF Q4_K_M)	~4.4 GB — без пика fp16 при загрузке, рекомендован для T4
VRAM (4-bit NF4 transformers)	~3.8 GB весов + ~1.8 GB overhead = ~5.6 GB
Температура генерации	0.05 (почти детерминированный режим для точных ответов)
Репетиционный штраф	repeat_penalty=1.1 (снижает повторения)
Среднее токенов на промпт	~900 (top-5 parent-чанков + системный промпт)

3.5.1 Три режима загрузки LLM

Ноутбук rag_vipnet_master.ipynb поддерживает три режима через переменную LLM_BACKEND:

Режим	Библиотека	VRAM	Скорость	Рекомендация
'gguf'	llama-cpp-python	~4.4 GB	Высокая	Рекомендован для Colab T4
'offload'	transformers + bitsandbytes	~5.6 GB GPU + ~8 GB RAM	Средняя	При нехватке VRAM
'small'	transformers + bitsandbytes	~3.4 GB	Очень высокая	Быстрый прототип

3.5.2 Системный промпт

Системный промпт жёстко ограничивает модель использованием только предоставленного контекста, исключая галлюцинации:

```
Ты — точный технический помощник по документации ViPNet Coordinator HW 5.
Твоя задача — отвечать на вопросы ИСКЛЮЧИТЕЛЬНО на основе предоставленных фрагментов.

Правила:
1. Используй только информацию из предоставленного контекста. Не домысливай.
2. Если ответа в контексте нет — скажи: «В предоставленных материалах ответа не
найдено».
3. При наличии синтаксиса команды, числа или алгоритма — обязательно приводи их.
4. Отвечай на русском языке, кратко и по существу.
5. Если контекст содержит предупреждения (Внимание/Примечание) — включай их в ответ.
```

4. Исследования и метрики

4.1 Метрики оценки

Качество системы оценивалось по четырём метрикам на ручном QA-бенчмарке из 50 вопросов.

Метрика	Формула	Что измеряет
Recall@k	I(gold ∈ top-k) / |Q|	Доля вопросов, для которых правильный чанк вошёл в top-k результатов
MRR	Σ (1 / rank_gold) / |Q|	Средний обратный ранг правильного чанка; учитывает его позицию
Context-F1	F1(tokens(top-1 child), tokens(answer))	Качество текста первого найденного чанка как ответа без LLM
Answer F1 (E2E)	F1(tokens(LLM-ответ), tokens(gold-ответ))	Token-F1 между ответом LLM и эталонным ответом; занижен для reasoning-вопросов

4.2 Результаты бенчмарка retrieval

Метод	MRR	Recall@1	Recall@3	Recall@5	Context-F1
TF-IDF baseline	0.364	0.28	0.48	0.50	0.127
BM25	0.469	0.38	0.62	0.68	0.168
Dense (bge-m3)	0.421	0.32	0.58	0.62	0.155
Hybrid BM25+bge-m3 RRF	0.549	0.46	0.68	0.72	0.214

Hybrid-метод превосходит baseline на 51% по MRR (0.549 vs 0.364) и на 44% по Recall@5 (0.720 vs 0.500).

4.3 Анализ по типам вопросов

Тип вопроса	Кол-во	Recall@3	Наблюдение
factoid	15	0.67	Средний результат; конкретные факты находятся хорошо
definition	10	0.60	Ниже среднего; помогает bge-m3 за счёт семантики
table	8	0.62	Таблицы хранятся целиком, поиск по числовым данным стабилен
reasoning	10	0.80	Лучший результат: уникальные технические термины — точные BM25-совпадения
procedural	7	0.57	Худший результат: шаги мастера разбиты по отдельным чанкам

4.4 Анализ по сложности вопросов

Сложность	Кол-во	Recall@3	Объяснение
easy	20	0.70	Прямые вопросы с ключевыми словами из документации
medium	22	0.61	Требуют понимания контекста

hard	8	0.88	Уникальные термины в сложных вопросах дают точные совпадения BM25

4.5 End-to-End оценка (10 вопросов)

E2E-оценка запускает полный пайплайн: retrieval + генерация LLM + сравнение с эталонным ответом по Token-F1.

ID	Тип	R@5	Answer F1	Комментарий
F07	factoid	1.0	0.778	Лучший результат: точный ответ на синтаксис команды
F08	factoid	1.0	0.706	Числовой факт найден и воспроизведён точно
D01	definition	1.0	0.500	Определение найдено, но перефразировано LLM
F09	factoid	0.0	0.082	Semantic gap: нужный чанк есть в корпусе, но не найден
R02	reasoning	1.0	0.000	Верный смысл ответа, но формулировка отличается — Token-F1 = 0

Вывод по E2E-оценке
Средний Answer F1 = 0.30. Метрика занижена: Token-F1 сравнивает токены буквально.
Ответы с верным смыслом, но иными формулировками (как R02) получают F1 = 0.
При R@5 = 1.0 фактическое качество ответа существенно выше показаний метрики.

5. Примеры реальных диалогов с системой

Все примеры получены в ходе реального запуска системы на документации ViPNet Coordinator HW 5.

5.1 Определение понятия

Вопрос	что такое VipNet

Ответ системы	VipNet — это зарегистрированный товарный знак компании АО «ИнфоТеКС». Это система управления сетевым взаимодействием.

Комментарий: система корректно нашла определение из документации и сформулировала краткий ответ в одном предложении. Тип вопроса — definition, Gold-чанк найден с R@5 = 1.0.

5.2 Процедурный вопрос (пошаговые инструкции)

Вопрос	Что нужно, чтобы запустить систему защиты от сбоев на одиночном узле?

```
Ответ:
Чтобы запустить систему защиты от сбоев на одиночном узле, необходимо:
1. Переключить систему в режим одиночного узла:
      hostname# failover config mode single
2. Отключить один из узлов кластера от локальной сети.
3. Убедиться, что IP-адреса сетевых интерфейсов второго узла заданы верно.

Источники (4):
  1. [code_example] Настройка CLI › Чтобы запустить систему защиты...  (score=0.032)
  2. [text        ] Настройка CLI › По умолчанию система защиты от...  (score=0.0318)
  3. [text        ] Настройка CLI › Назначение системы защиты от сб... (score=0.0315)
  4. [code_example] Настройка CLI › Переключение системы защиты от... (score=0.0302)
```

Комментарий: система вернула структурированный ответ с синтаксисом команды и пошаговым алгоритмом. Найдено 4 релевантных источника из раздела «Настройка CLI», включая code_example с точной командой.

5.3 Вопрос вне области знаний системы

Вопрос	Как попасть на стажировку?

```
Ответ:
B所提供的文档材料中没有关于如何进行实习的信息 (В предоставленных материалах ответа не на
йдено)

Источники (5):
  1. [command] Справочник команд › machine set loghost  (score=0.0164)
  2. [text   ] Настройка CLI › Для каждого резервируемого инте... (score=0.0164)
  ...
```

Комментарий: система корректно отказала в ответе, поскольку информации о стажировках в технической документации ViPNet нет. Системный промпт с инструкцией «не домысливай» предотвратил галлюцинацию. Retrieval нашёл несвязанные фрагменты с низкими score (~0.016), что является признаком отсутствия релевантного контекста. Однако, модель ответила на другом языке, что при исследовании было списано на перекос в изначальных обучающих данных модели.

6. Описание входных данных

6.1 Корпус документов

Файл	Документ	Тип	Child-чанков	Основное содержание
01_txt.txt	Подготовка к работе	user_guide	~700	Инициализация, роли, мастер настройки
02_txt.txt	Настройка CLI	cli_guide	~800	Командный интерпретатор, конфигурация сети
03_txt.txt	Веб-интерфейс	web_guide	~600	Управление через браузер
04_txt.txt	Справочник команд	command_ref	~1 600	Синтаксис CLI-команд (наибольший объём)
06_txt.txt	История версий	release_notes	~400	Changelog по версиям
07_txt.txt	Трансиверы	hardware	~400	Аппаратные характеристики, таблицы
ИТОГО	6 документов / ~1 200 стр.	—	~4 500	—

6.2 QA-бенчмарк

Тип вопроса	Кол-во	easy	medium	hard	Пример вопроса
factoid	15	7	5	3	«Каков синтаксис команды admin ssh show-key?»
definition	10	4	4	2	«Что такое TCP-туннель в контексте ViPNet?»
table	8	4	3	1	«Сколько ядер для Coordinator VA500 с DPI?»
reasoning	10	3	4	3	«Почему TCP-туннель используется вместо UDP?»
procedural	7	1	4	2	«Как задать шлюз при инициализации?»
ИТОГО	50	20	22	8	—

7. Возможные улучшения

7.1 Retrieval

Улучшение	Ожидаемый эффект
Cross-Encoder reranking (BAAI/bge-reranker-v2-m3) Схема: Hybrid top-20 → reranker → top-5 → LLM	+5–10% MRR
Переформулировать 2–3 варианта запроса, объединение через RRF	+5–10% Recall
Исправление детектора оглавления (02_txt, 03_txt) 2 429 чанков с некорректным маркером	+3–5% MRR

7.2 Генерация

- Qwen2.5-14B-Instruct при наличии A100/H100: ожидаемый прирост +10–20% Answer F1.
- Примеры ответов в промпте: улучшает стабильность форматирования для команд CLI и таблиц.
- Добавить рассуждение модели для reasoning-вопросов: добавление «Объясни пошагово» в промпт.

7.3 Чанкинг

- Снижение CHUNK_MIN_CHARS с 80 до 50 символов: вернёт короткие информативные фрагменты.
- Специализированный чанкинг для процедурных инструкций: сохранять смежные нумерованные шаги вместе (главная слабость — Recall@3 = 0.57 для procedural).
- Улучшить дедупликацию: текущее сравнение первых 150 символов не ловит семантически идентичные чанки.

7.4 Оценка

- RAGAS-метрики (Faithfulness, Answer Relevancy, Context Precision) вместо Token-F1 — не зависят от буквальных совпадений слов, а от передаваемого смысла.
- BERTScore для семантической близости ответов.
- Расширение бенчмарка до 200–300 вопросов с смежными по категориям вопросами для снижения дисперсии и уточнения реального качества модели.

8. Заключение

В рамках работы разработана и экспериментально оценена RAG-система для автоматического ответа на вопросы по документации ViPNet Coordinator HW 5. Система реализует полный пайплайн от обработки документов до генерации ответов на естественном языке с указанием источников.

Ключевые результаты:

1. Иерархический чанкинг с разделением на Parent (генерация) и Child (поиск) обеспечивает эффективный компромисс между точностью поиска и полнотой контекста для LLM.
2. Гибридный поиск (BM25 + BAAI/bge-m3 + RRF) даёт наилучший результат: MRR = 0.549, Recall@5 = 0.720 — превосходя каждый из компонентов в отдельности.
3. Основной источник ошибок — семантические разрывы (17/50 вопросов), устранимые через Cross-Encoder reranking.
4. Qwen2.5-7B-Instruct через GGUF успешно работает на Tesla T4, генерируя качественные ответы для большинства типов вопросов.
5. Система корректно отклоняет вопросы вне области знаний, отвечая «В предоставленных материалах ответа не найдено».

Итоговые метрики системы
Лучший retrieval (Hybrid BM25+bge-m3): MRR = 0.549 · Recall@5 = 0.720 · Context-F1 = 0.214
E2E генерация (10 вопросов): Avg Answer F1 = 0.30 · Context Found Rate = 0.80
Репозиторий: https://github.com/maratsaratov/tech-task_infotecs